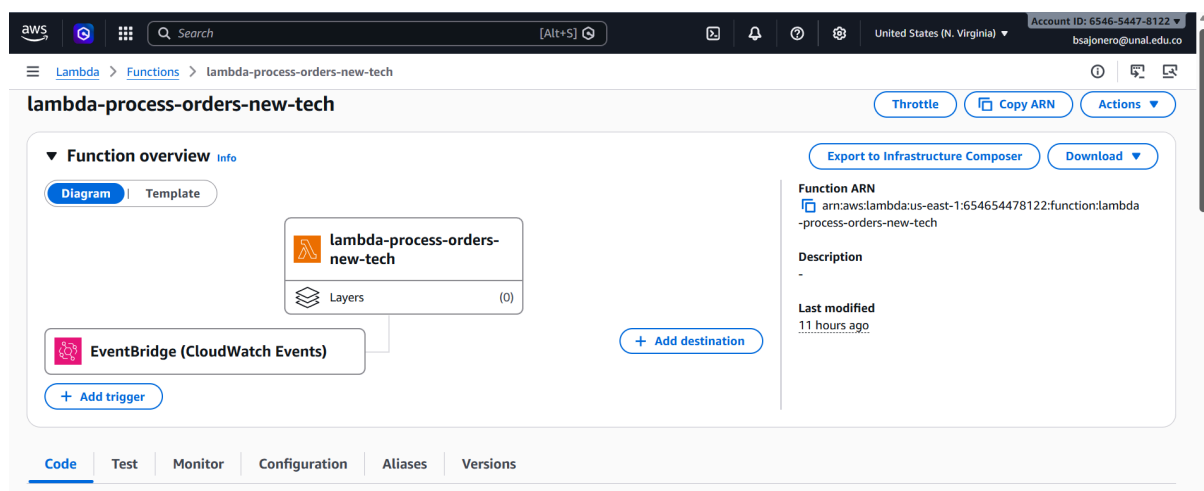


Examen Práctico - Observabilidad y Troubleshooting de AWS Lambda

Postmortem Técnico

1. Resumen del incidente

Se identificó que la función Lambda **lambda-process-orders-new-tech** presenta un comportamiento anómalo e intermitente en su ejecución. El incidente se manifiesta principalmente como un retraso crítico en el procesamiento de órdenes asíncronas, donde los eventos sufren una alta retención en la cola interna de AWS antes de poder ser ejecutados con éxito, afectando la disponibilidad del flujo de procesamiento.



2. Tipos de errores encontrados

A través del análisis detallado, se determinó que la función no sufre de un único problema, sino de una combinación de fallas de código y de infraestructura:

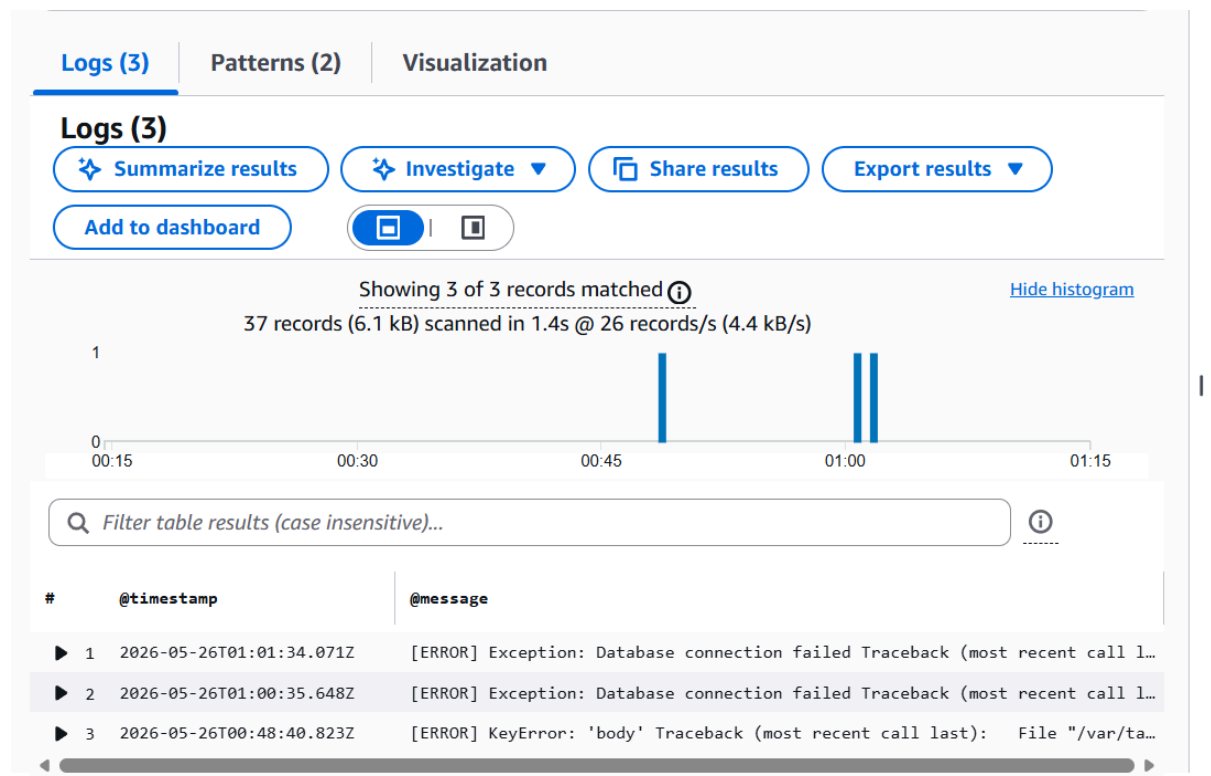
- **KeyError ('body')**: Error de ejecución/lógica de la aplicación. La función intenta acceder a una propiedad inexistente dentro del payload de la orden recibida, provocando que la ejecución falle inmediatamente al iniciar.
- **Exception (Database connection failed)**: Error de infraestructura y dependencias. La función se ejecuta, pero no logra establecer comunicación con el backend de la base de datos, lo que genera un *timeout* de red externo y provoca que AWS reintente procesar la orden automáticamente de manera asíncrona.

3. Análisis Técnico Detallado

A. Análisis de Logs (Amazon CloudWatch Logs)

Utilizando *CloudWatch Logs Insights*, se escanearon los registros históricos del grupo de logs `/aws/lambda/lambda-process-orders-new-tech`, arrojando las siguientes conclusiones:

1. **Identificación de errores:** Se aislaron excepciones explícitas de Python de tipo `KeyError: 'body'` y fallas de conexión de base de datos (`Database connection failed Traceback`).
2. **Frecuencia:** Los errores se concentran en ráfagas específicas (ej. entre las `00:48` y las `01:01`).
3. **Clasificación:**
 - **Timeouts:** No se observaron *timeouts* de la función Lambda en sí (no llegó al límite de sus segundos configurados de ejecución global), pero sí existen *timeouts* de conexión de red hacia la base de datos.
 - **Exceptions / Errores de ejecución:** El 100% de los fallos en los logs corresponden a excepciones no controladas en el código de la aplicación.

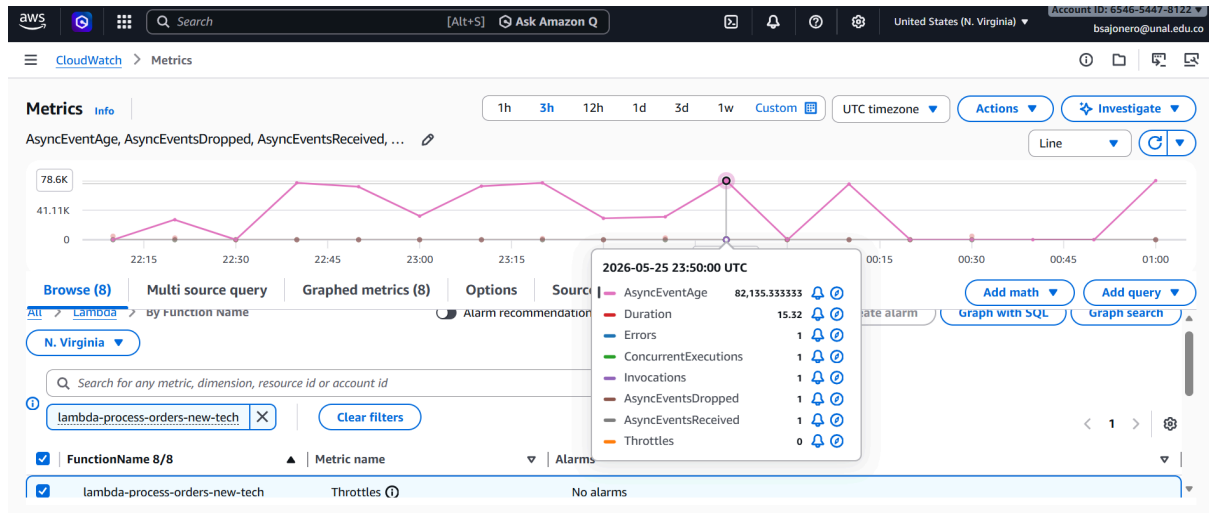


B. Métricas CloudWatch (CloudWatch Metrics)

1. **Número de invocaciones y comportamiento de Errors:** El volumen de invocaciones es bajo e intermitente (picos aislados de 1 petición por intervalo). Sin embargo, la métrica de `Errors` se activa de forma síncrona con cada fallo de conexión.
2. **Comportamiento de la duración (Duration) y Latencia:** La latencia interna de procesamiento es baja cuando ocurre el `KeyError` debido a que la función "falla

rápido" (fail-fast) en apenas unos milisegundos. No obstante, existe un incremento masivo en la latencia de entrega debido a la métrica **AsyncEventAge**.

3. **Métrica Clave Anómala:** La métrica **AsyncEventAge** escala drásticamente hasta alcanzar picos de **82.22K**. Esto demuestra que el verdadero cuello de botella es la acumulación de eventos envejecidos en la cola interna de AWS debido a los ciclos de reintentos (*retries*) que gatillan los fallos de la base de datos.



C. Análisis con X-Ray (AWS X-Ray)

1. **Traces con errores y comportamiento general:** Al analizar el *Trace Map* en un rango de 1 hora, X-Ray calcula de forma automática una **tasa de error del 33% (Errors 4xx)** sobre el total de las peticiones capturadas.
2. **Análisis de Fault o Exception:** X-Ray clasifica estas fallas como errores de la serie 4xx (cuadro amarillo en la consola) debido a que las excepciones son generadas por la lógica interna de la aplicación (el **KeyError** de Python) y el manejo de los datos de entrada, manteniendo una latencia promedio general de **565 ms**.
3. **Tiempos de ejecución elevados:** Las peticiones individuales no muestran degradación de cómputo en la Lambda, confirmando que el problema de rendimiento es estrictamente externo (tiempos de espera de conexión de red e intervalos de reintento asíncrono).

